

《操作系统原理》 实验

三、线程的调度

重庆大学软件学院 洪明坚
2016年4月

目录

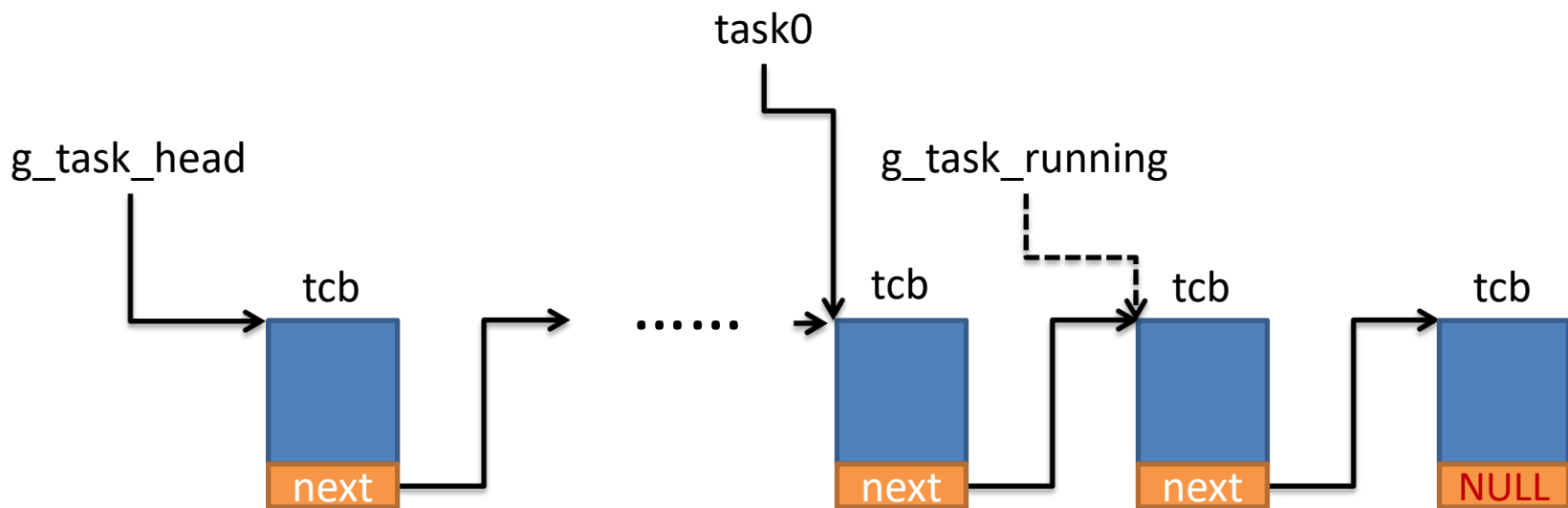
- 掌握线程的调度
 - 优先级调度†
- FAQ

基本概念

- 优先级
 - 静态优先级(nice): 内核不会修改它，不随时间而变化，除非用户通过系统调用setpriority进行修改
 - 动态优先级(priority): 内核根据线程使用CPU的状况、静态优先级nice和系统负荷计算出来，会随时间而变化
 - 最终的调度依据，即调度器只根据动态优先级进行调度

数据结构

- 单向链表



- 注意

- 1、在系统启动过程中，**g_task_running**是**NULL**！
- 2、task0(tid=0)是系统空闲线程（system idle thread）

实验步骤

- Step1: 在“**struct tcb**”中增加线程的静态优先级**nice**
 - 一定要加在**kstack**字段之后、**signature**字段之前!!!
 - 在函数**sys_task_create**中初始化**nice=0**
 - **nice**是整数，取值范围**[-NZERO, NZERO-1]**，值越小优先级越高
 - **#define NZERO 20**

实验步骤（续）

- Step2: 增加系统调用
 - `int getpriority(int tid)`
 - 成功返回线程tid的(nice+NZERO)，失败返回-1
 - `int setpriority(int tid, int prio)`
 - 把线程tid的nice设为(prio-NZERO)
 - prio必须在[0,2*NZERO-1]内
 - 成功返回0，失败返回-1
 - 注意
 - 如果参数tid=0，表示获取/设置当前线程的nice值，而不是task0！
 - 函数“`struct tcb *get_task(int tid)`”用于根据tid获取struct tcb的指针。调用时一定要用save_flags_cli/restore_flags保护起来

```
uint32_t flags; struct tcb *tsk;  
save_flags_cli(flags);  
tsk = get_task(tid);  
restore_flags(flags);
```

实验步骤（续）

- Step3: 在“**struct tcb**”中，再增加两个属性
 - estcpu: 表示线程最近使用了多少CPU时间
 - 在函数sys_task_create中初始化estcpu=0
 - 每次定时器中断: g_task_running->estcpu++, **task0除外!**
 - 每秒钟为所有线程（运行、就绪和等待）更新一次
$$estcpu = \frac{2 \times g_load_avg}{2 \times g_load_avg + 1} \times estcpu + nice$$
 - priority: 表示线程的动态优先级
 - priority = PRI_USER_MAX-(estcpu/4)-(nice*2)
 - 截断到PRI_USER_MIN到PRI_USER_MAX范围内
 - » #define PRI_USER_MIN 0
 - » #define PRI_USER_MAX 127
 - 值越**大**优先级越**高**

实验步骤（续）

- Step4: 增加一个全局变量
 - g_load_avg: 表示系统的平均负荷
 - 初值为0
 - 每秒钟更新一次
 - $g_load_avg = (59/60) \times g_load_avg + (1/60) \times nready$
 - » nready表示处于就绪状态的线程个数，task0除外！

实验步骤（续）

- Step5: 属性计算
 - g_load_avg 和线程的estcpu
 - 在定时器的中断处理函数（ISR）中计算
 - 文件timer.c中的函数isr_timer
 - 如何每隔一秒计算一次？
 - (g_timer_ticks % HZ)
 - » =0, 表示1秒钟已经过去
 - » 否则, 还不到1秒钟

实验步骤（续）

- Step6: 修改task.c中的函数schedule，实现优先级调度算法
 - 先计算各个线程的动态优先级priority（task0除外！），然后进行调度
 - 注意：
 - 线程0（task0，它的ID=0）是一个特殊的线程，仅当没有其他可运行的线程时，才能调度task0运行！
 - 函数schedule被执行时，CPU的中断已经被关闭。

实验步骤（续）

- 性能问题
 - nice , priority和nready都是整数
 - 性能不是问题
 - g_load_avg 和estcpu是实数
 - 浮点(float-point)表示：精度高，性能差！†
 - 定点(fixed-point)表示：精度低，性能好！
 - 文件fixedptc.h中定义了定点数类型fixedpt及其运算
 - 例子
 - » fixedpt g_load_avg;
 - » struct tcb { ... fixedpt estcpu; ...};

†内核一般不允许浮点运算，但支持应用程序进行浮点运算

实验步骤（续）

- 性能问题（续）

- `g_task_running->estcpu++`

- `g_task_running->estcpu = fixedpt_add(g_task_running->estcpu, FIXEDPT_ONE);`

- `g_load_avg = (59/60)*g_load_avg+(1/60)*nready`

- `fixedpt r59_60 = fixedpt_div(fixedpt_fromint(59), fixedpt_fromint(60));`

- `fixedpt r01_60 = fixedpt_div(FIXEDPT_ONE, fixedpt_fromint(60));`

- `g_load_avg = fixedpt_add(fixedpt_mul(r59_60, g_load_avg),
fixedpt_mul(r01_60, fixedpt_fromint(nready)));`

- `priority = PRI_USER_MAX-(estcpu/4)-(nice*2)`

- `priority = PRI_USER_MAX -`

- `fixedpt_toint(fixedpt_div(estcpu, fixedpt_fromint(4))) -`

- `p->nice*2;`

实验步骤（续）

- 性能问题（续）

$$estcpu = \frac{2 \times g_load_avg}{2 \times g_load_avg + 1} \times estcpu + nice$$

```
fixedpt ratio;
```

```
ratio = fixedpt_mul(FIXEDPT_TWO, g_load_avg);
```

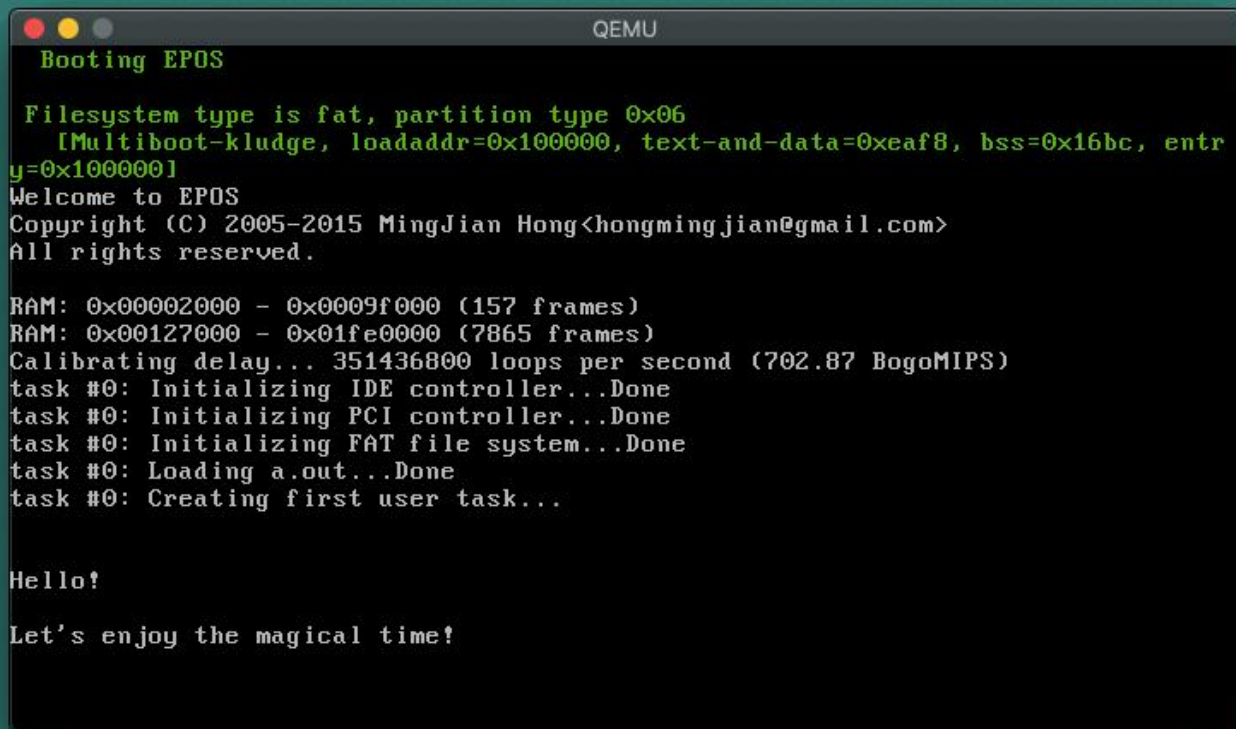
```
ratio = fixedpt_div(ratio, fixedpt_add(ratio, FIXEDPT_ONE));
```

```
estcpu = fixedpt_add(fixedpt_mul(ratio, estcpu),  
                     fixedpt_fromint(nice));
```

实验步骤（续）

- Step5: 测试调度器
 - 创建两个冒泡排序的线程，记为A和B
 - 分别占用屏幕的两个位置
 - 在屏幕上用进度条动态显示两个线程的静态优先级
 - 排序线程的速度不能太快或太慢，否则无法控制优先级
 - 用[nanosleep](#)让线程暂停运行一段时间
 - 另外创建一个控制线程
 - 循环等待键盘输入
 - `int key = getchar();`
 - 如果`key=0x4800(up)/0x5000(down)`
 - 调用`setpriority`调高/低A线程的静态优先级
 - 更新A的优先级进度条
 - 如果`key=0x4d00(right)/0x4b00(left)`
 - 调用`setpriority`调高/低B线程的静态优先级
 - 更新B的优先级进度条
 - 把控制线程的静态优先级设置到最高，以保证控制效果

效果展示



```
QEMU
Booting EPOS

Filesystem type is fat, partition type 0x06
[Multiboot-kludge, loadaddr=0x100000, text-and-data=0xeaf8, bss=0x16bc, entry=0x100000]
Welcome to EPOS
Copyright (C) 2005-2015 MingJian Hong<hongmingjian@gmail.com>
All rights reserved.

RAM: 0x00002000 - 0x0009f000 (157 frames)
RAM: 0x00127000 - 0x01fe0000 (7865 frames)
Calibrating delay... 351436800 loops per second (702.87 BogoMIPS)
task #0: Initializing IDE controller...Done
task #0: Initializing PCI controller...Done
task #0: Initializing FAT file system...Done
task #0: Loading a.out...Done
task #0: Creating first user task...

Hello!

Let's enjoy the magical time!
```

目录

- 掌握线程的调度
 - 优先级调度
- FAQ

FAQ

- Q1: 静态优先级的名字为什么是nice?
 - A1: 看[这里](#)。
- Q2: 如何打印fixedpt类型变量的值，比如g_load_avg?
 - A2:

```
char num[20];  
fixedpt_str(g_load_avg, num, -2);  
printf("g_load_avg = %s\r\n", num);
```

提交实验

- 命令：make submit
 - 自动检测修改过的源文件，并压缩后上传
 - 注意：
 - 确保计算机连接到互联网
 - 压缩后不能超过64KiB
 - 只允许提交一次
 - 在截止时间之前，如果需要重复提交，请线下联系
 - 看到 “Successfully submitted.” 才算提交成功
 - 过期后果自负
 - 如果有特殊原因，需提供有效的证明材料

That's all
Enjoy hacking